Retrospectiva del Proyecto Inicial: Slot Machine (Ciclos 1 a 4 y Cierre)

Autores: Juan David Guarguati Guarguati y Juan Pablo Suárez Rubiano

---

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Los mini-ciclos se definieron por **requisito funcional**. Cada uno termina con código que compila y con pruebas que pasan, así que siempre hubo una versión funcionando para mostrar.

Ciclo 1: simulador básico

Mini-ciclo	Contenido	Justificación
1.1	Crear la máquina, `Symbol` y `Wheel` (Req. 1)	Es la base de todo lo demás.
1.2	Adicionar y eliminar ruedas y símbolos (Req. 2 y 3)	Sin ruedas y símbolos no hay nada que girar.
1.3	Girar y consultar (Req. 4 y 5)	Es el comportamiento principal del simulador.
1.4	Jackpot (Req. 6)	Depende de poder consultar los símbolos.
1.5	Visibilidad, terminar y mensajes de error (Req. 7 y 8, usabilidad)	Se puede trabajar invisible y luego añadir el dibujo.

Ciclo 2: nuevas operaciones

Mini-ciclo	Contenido	Justificación
2.1	`swap` (Req. 9)	Es independiente de los demás.
2.2	`lock` / `unlock` (Req. 10)	Los giros siguientes deben respetar el bloqueo.
2.3	`spin(wheel)` y `spin(wheel, steps)` (Req. 11)	Usan el bloqueo del mini-ciclo anterior.
2.4	`spin(String[])`, `placeSymbol`, `symbols`, `distinctSymbols` (Req. 12)	Consultas y colocación directa.
2.5	Pruebas `SlotMachineC2Test` y `SlotMachineCC2Test`	Para cerrar el ciclo con pruebas.

Ciclo 3: resolver la maratón

Mini-ciclo	Contenido	Justificación
3.1	`SlotMachine(n)` aleatoria, con n entre 3 y 50 (Req. 13)	Es el problema a resolver.
3.2	`solve(n)` usando solo `SlotMachine(n)`, `spin(wheel, steps)` y `distinctSymbols()` (Req. 14)	Es la restricción de diseño del ciclo.
3.3	`simulate(n)` con `makeVisible()` (Req. 15)	Hace lo mismo que `solve`, pero a la vista.
3.4	Paleta de 50 colores, marco de jackpot, pruebas	Para que n = 50 se vea y se pruebe.
3.5	Corrección: el Contest ya no usa `ok()`; valida n con `MIN_N`/`MAX_N`; toda la lógica queda dentro de `solve` y `simulate`	`ok()` no estaba entre los métodos permitidos, y así cada método se lee de principio a fin.

Ciclo 4: refactorización y extensión

Mini-ciclo	Contenido	Justificación
4.1	**Refactorizar:** `Wheel` y `Symbol` abstractas, `NormalWheel` y `NormalSymbol`, fábricas `create`	Sin cambiar el comportamiento, comprobado con las pruebas de los ciclos 2 y 3 antes de extender.
4.2	`addWheel(type, pos)`, `addSymbol(type, pos, color)` y símbolos registrados en la máquina (Req. 16)	Todo lo demás se crea con estos métodos; y las pruebas compartidas agregan símbolos antes que ruedas.
4.3	Ruedas `lefty` y `rebel` (Req. 17); `swap` con toda su lógica adentro	Cada tipo es una subclase que redefine un solo punto. `swap` perdía los tipos.
4.4	Símbolos `ephemeral` y `shy` (Req. 18)	Igual, con los ganchos `spun()` y `selected()`.
4.5	Rueda propuesta `reverse` (Req. 19)	Muestra que extender ya no cuesta: una clase y una línea en la fábrica.
4.6	Figuras y marcas por tipo (Usabilidad 1)	Cada tipo se ve distinto.
4.7	Pruebas C4, CC4 y de aceptación, más la documentación y el Astah	Para cerrar el ciclo con pruebas y diseño.
4.8	**Herencia en `shapes`** (sugerencia del profesor): clase abstracta `Shape`, de la que heredan `Rectangle`, `Circle` y `Triangle`;	Quita el código repetido de las figuras y de los símbolos: mover, mostrar y ocultar se escriben una sola vez.

Mini-ciclo	Contenido	Justificación
	`Symbol` guarda su figura como `Shape`	
4.9	**Correcciones del Astah:** atributos y métodos `static` subrayados, los `ArrayList` de clases propias dibujados como composición, 5 diagramas de secuencia que faltaban (`spin(wheel)`, `spin(String[])`, `delSymbol`, `distinctSymbols`, `isJackpot`), 4 de los tipos (`afterSpin`, `spun`, `rotate`, `selected`) y un actor en todos los diagramas	El diseño tiene que coincidir con el código y seguir la notación UML.

Cierre (Ciclo 5): calidad con herramientas modernas

Mini-ciclo	Contenido	Justificación
5.1	Pasar el proyecto a **VS Code** (`src/main/java`, `src/test/java`, JUnit en `lib/`, configuración en `.vscode`)	VS Code reconoce el proyecto solo, sin herramientas extra, y deja correr las pruebas, con y sin cubrimiento, desde el panel Testing.
5.2	Tomar los **resultados iniciales** de JaCoCo y PMD sin cambiar nada	Para tener con qué comparar y decidir qué arreglar.
5.3	Crear **`SlotMachineC1Test`** y **`SlotMachineC3Test`**	Lo pide la entrega (`SlotMachine*Test` 1, 2, 3, 4 y Contest), y además cubren casos de error que no se probaban.
5.4	Corregir las **11 violaciones de prioridad alta** de PMD	Es la meta del análisis estático.
5.5	**Documentación inicial de las clases más corta**, con `@author` y `@version`	Seguir el estándar de documentación de Java: una frase clara por clase.
5.6	**Resultados finales**, informes, retrospectiva y zip de entrega	Para cerrar el proyecto.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

Ciclo	Estado	Detalle
1	Terminado (1.1 a 1.5)	Todos los requisitos funcionan, en modo visible e invisible.

Ciclo	Estado	Detalle
2	Terminado (2.1 a 2.5)	Las 14 pruebas de C2 y CC2 siguen pasando en el Ciclo 4 sin cambios.
3	Terminado (3.1 a 3.5)	`solve` es determinístico, usa a lo sumo un spin por rueda y funciona hasta n = 50.
4	Terminado (4.1 a 4.9)	Pasan las 73 pruebas de unidad del ciclo y las 2 de aceptación se ejecutaron completas. La herencia en `shapes` no cambió nada de lo que se ve. El Astah quedó al día con el código: diagrama de clases corregido y todos los diagramas de secuencia con actor. 4 pruebas compartidas de otros grupos quedaron comentadas porque usan clases internas incompatibles con nuestro diseño (el motivo está en `SlotMachineCC4Test`).
Cierre	Terminado (5.1 a 5.6)	El proyecto abre y corre en VS Code. Pasan las **99 pruebas de unidad**, y la de aceptación se ejecuta desde VS Code. **JaCoCo:** 85,1 % inicial y **87,7 %** final (meta: más de 75 %). **PMD:** 11 violaciones de prioridad alta al inicio y **0** al final.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

Ciclo	Juan David Guarguati	Juan Pablo Suárez
1	12	12
2	12	12
3	12	12
4	12	12
Cierre	12	12
Total	**60**	**60**

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

- **En el proyecto:** resolver el problema de la maratón “a ciegas” (Ciclo 3). Se usa solo `distinctSymbols()` como medida, y el resultado es determinístico, con un solo spin por rueda.

- **En el Cierre:** comprobar con herramientas la calidad del simulador. Las pruebas cubren el 87,7 % del código y no queda ninguna regla de prioridad alta de PMD sin cumplir.
- **En el Ciclo 4:** reorganizar el proyecto con herencia sin dañar lo que ya funcionaba. `SlotMachine` ya no conoce las clases concretas, así que un tipo nuevo (por ejemplo la rueda `reverse`) se agrega con una clase y una línea en la fábrica, sin tocar la máquina. Esto cumple el requisito de extensibilidad.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

- **Ciclo 3:** resolver sin leer los colores. Se resolvió comparando la suma de `distinctSymbols()` en vueltas completas: la diferencia correcta da la suma mínima.
- **Ciclo 4, qué es un "giro":** el enunciado no lo dice, y de eso dependen la lefty, el ephemeral y el shy. Se definió que giran `spin()`, `spin(wheel)` y `spin(wheel, steps)`, mientras que `placeSymbol` y `spin(String[])` solo seleccionan. Los efectos del giro quedaron en un solo método, `finishSpin`.
- **Ciclo 4, pasar las pruebas compartidas:** otros grupos agregan símbolos antes que ruedas, esperan que un shy escondido no cuente para el jackpot y que la lefty copie todo el estado de su vecina. Se resolvió guardando los símbolos registrados en la máquina, copiándolos a cada rueda nueva y haciendo que la lefty copie también si el shy está escondido.
- **Ciclo 4, `swap`:** la versión anterior copiaba los colores de una rueda a otra y creaba símbolos nuevos, así que se perdían los tipos. Ahora `swap` intercambia las ruedas completas, con toda la lógica dentro del método.
- **Ciclo 4, `exit()`:** llamaba a `System.exit(0)`, que cerraba BlueJ y cortaba las pruebas compartidas que llaman `exit()` al final. Ahora solo oculta el simulador.
- **Ciclo 4, herencia en `shapes`:** el nombre `Shape` chocaba con `java.awt.Shape` dentro de `Canvas`. Se resolvió escribiendo `java.awt.Shape` con su nombre completo en `Canvas`.
- **Cierre, PMD:** el constructor de `Wheel` llamaba a `styleMarker(...)`, un método abstracto, antes de que la subclase terminara de construirse. Se resolvió pasando esa llamada a la fábrica `Wheel.create(...)`. También se hicieron abstractos los métodos vacíos de `Symbol` (`spun`, `selected`, `setHidden`).
- **Cierre, pruebas con pantalla:** `simulate(n)` abre el lienzo y muestra el aviso de JACKPOT, así que esa prueba necesita pantalla. La prueba de aceptación se dejó fuera del cubrimiento porque depende de que una persona responda.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Mantuvimos una buena comunicación y nos repartimos las tareas de forma ordenada; luego las integramos al proyecto entre los dos.

Nos comprometemos a actualizar el diagrama de Astah al terminar cada mini-ciclo y no al final, para evitar complicaciones de última hora. También a correr JaCoCo y PMD desde el inicio de cada proyecto, y no solo al cierre.

7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿Por qué?

La **refactorización apoyada en pruebas**. Antes de agregar los tipos nuevos cambiamos la estructura de `Wheel` y `Symbol`, y las pruebas de los ciclos anteriores nos confirmaron que todo seguía funcionando igual. Así pudimos extender el proyecto con confianza. También ayudó el **diseño simple**: cada tipo redefine solo lo que cambia. En el cierre volvió a servir: cada corrección de PMD se comprobó corriendo otra vez las 99 pruebas.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil?

- Barnes, D. J., & Kölling, M. (2016). *Objects First with Java: A Practical Introduction Using BlueJ* (6th ed.). Pearson. (Proyecto `shapes`.)
- Beck, K. (2000). *Extreme Programming Explained: Embrace Change*. Addison-Wesley.
- Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley.
- Oracle. (s.f.). *Abstract Methods and Classes (The Java Tutorials)*. https://docs.oracle.com/javase/tutorial/java/landl/abstract.html
- Oracle. (s.f.). *Inheritance (The Java Tutorials)*. https://docs.oracle.com/javase/tutorial/java/landl/subclasses.html
- Oracle. (s.f.). *How to Write Doc Comments for the Javadoc Tool*. https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html
- ICPC. (2025). *Problem I: Slot Machine*. ICPC World Finals.
- JaCoCo. (s.f.). *JaCoCo Java Code Coverage Library*. https://www.jacoco.org/jacoco/
- PMD. (s.f.). *Java Rules*. https://docs.pmd-code.org/latest/pmd_rules_java.html
- Microsoft. (s.f.). *Java in Visual Studio Code*. https://code.visualstudio.com/docs/languages/java
- Anthropic. (2026). *Claude*. Usado para revisar el diseño, implementar y verificar las pruebas. https://claude.ai

La más útil fue la guía de Oracle sobre clases abstractas, porque es la base de la refactorización de este ciclo.